

Explicarea functionarii codului proiectului CRM Training IT

Textul de mai jos prezinta functionarea codului proiectului, arhitectura, fluxurile principale si modul in care sunt aplicate design pattern-urile.

1. Introducere: ce este proiectul CRM Training IT

Proiectul CRM Training IT este o aplicatie Java realizata pentru contextul unei firme care vinde cursuri IT. Scopul ei este sa centralizeze informatii despre persoane interesate de cursuri, companii care pot cumpara traininguri corporate, activitati comerciale, oportunitati B2B, cursuri, sesiuni si inscrieri. Din punct de vedere al codului, proiectul nu este un simplu CRUD, desi are multe operatii de creare, cautare si actualizare. El este gandit ca o demonstratie de arhitectura software, in care aceeasi problema de business este impartita in straturi clare si in componente care folosesc design patterns cunoscute.

Aplicatia este pornita din clasa `crm.CrmApplication` si ruleaza in consola. Interfata `ConsoleUI` prezinta un meniu cu operatii precum adaugarea unui lead individual, adaugarea unui client corporate, cautarea unui contact dupa email, listarea lead-urilor fierbinti, schimbarea statusului unui lead, adaugarea unei activitati, crearea unei oportunitati B2B si vizualizarea pipeline-ului activ. Acest UI de consola nu este partea principala a proiectului, ci un mod simplu de a demonstra functionalitatea. Partea importanta este structura interna: cum o comanda din meniu ajunge in servicii, cum este validata, cum se calculeaza scorul, cum se salveaza datele si cum se declanseaza evenimente secundare.

Domeniul ales este potrivit pentru un CRM deoarece are doua categorii naturale de clienti. Prima categorie este B2C: persoane individuale care cauta un curs, de exemplu Java Fundamentals sau Python Fundamentals. A doua categorie este B2B: companii care vor training pentru angajati. In loc sa creeze doua modele complet separate, autorul a ales o entitate comuna, `Contact`, care poate fi individuala sau corporate prin campul `contactType`. Aceasta decizie simplifica multe fluxuri, pentru ca un contact poate avea email, telefon, status, scor si activitati indiferent de natura lui.

Proiectul trebuie privit ca un nucleu de CRM. Nu exista interfata web, autentificare reala sau dashboard grafic, dar exista o baza de cod suficient de organizata pentru a putea fi extinsa. Daca s-ar dori transformarea proiectului intr-o aplicatie de productie, logica existenta din servicii, repository-uri, DAO-uri si modele ar putea fi pastrata, iar deasupra ei s-ar putea adauga un API REST si o interfata web. Din acest motiv, capitolul de fata explica mai ales cum functioneaza codul si cum se leaga componentele intre ele.

2. Tehnologii folosite si structura generala a proiectului

Proiectul este construit cu Maven, iar fisierul `pom.xml` defineste atat dependintele, cat si modul in care se creeaza JAR-ul executabil. Limbajul tinta este Java 8, ceea ce explica folosirea unor constructii precum `Optional`, `LocalDate`, `LocalDateTime` si interfete functionale simple. Pentru reducerea codului repetitiv se foloseste Lombok, mai ales prin anotari precum `Getter`, `Setter`, `NoArgsConstructor`, `AllArgsConstructor`, `Builder` si `ToString`. Astfel, clasele de model nu contin manual toate metodele de acces, dar la compilare acestea sunt generate.

Pentru logging se foloseste SLF4J impreuna cu Logback. Configuratia `logback.xml` trimite logurile atat in consola, cat si in fisierul `logs/crm-application.log`. Aceasta alegere este importanta deoarece proiectul foloseste multe layere si multe evenimente, iar logurile permit urmarirea fluxului real de executie. Din logurile existente se vede, de exemplu, ca aplicatia initializeaza pool-ul de conexiuni, inregistreaza observarii, executa comenzi si salveaza contacte in baza de date.

Pentru acces la baza de date se foloseste JDBC pur, fara Hibernate sau JPA. Conexiunile nu sunt create manual la fiecare operatie, ci sunt administrate prin HikariCP, un connection pool performant. Clasa `DatabaseConnection` construiesc un `HikariDataSource` pe baza proprietatilor incarcate de `AppConfig`. Implicit, aplicatia se conecteaza la MariaDB pe localhost, baza de date `crm_training`, cu user `root` si parola goala. Aceasta configuratie este utila pentru demo local, dar intr-un mediu real ar trebui mutata intr-un fisier de configurare extern si securizat.

Structura pachetelor arata intentia autorului. Pachetul `crm.ui` contine interfata de consola. Pachetul `crm.facade` contine `CrmFacade`, punctul simplificat de acces catre sistem. Pachetul `crm.service` contine logica de business impartita pe domenii: `contact`, `course`, `enrollment`, `opportunity`, `activity` si `notification`. Pachetele `crm.repository` si `crm.dao` se ocupa de accesul la date. Pachetul `crm.model` contine entitati si enumerari. Exista apoi pachete speciale pentru `command`, `observer`, `strategy`, `builder`, `factory` si `validation`. Aceasta organizare nu este intamplatoare, ci urmareste sa demonstreze aplicarea design patterns in zone concrete ale aplicatiei.

3. Pornirea aplicatiei si initializarea sistemului

Fluxul de pornire este concentrat in clasa `crm.CrmApplication`. Metoda `main` este punctul de intrare al programului si executa pasii principali necesari pentru ca sistemul sa fie pregatit. Mai intai se incarca `AppConfig`, care este un Singleton. Aceasta clasa citeste proprietatile aplicatiei din `application.properties` daca fisierul exista, dar are si valori implicite pentru dezvoltare. In cazul proiectului analizat, lipsa fisierului `application.properties` nu opreste aplicatia, ci determina folosirea configuratiei default.

Dupa configurare, se initializeaza `DatabaseConnection`. Aceasta clasa este tot un Singleton, dar implementat cu double-checked locking, adica instanta este creata o singura data, intr-un mod sigur pentru acces concurrent. Constructorul clasei construiesc un `HikariConfig`, seteaza URL-ul JDBC, userul, parola, driverul MariaDB si parametrii de pool, apoi creeaza `HikariDataSource`. Din acel moment, orice DAO poate cere o conexiune prin `DatabaseConnection.getInstance().getConnection()`. Conexiunea returnata este de fapt luata din pool si revine in pool cand este inchisa.

Un pas esential la pornire este inregistrarea observerilor in `EventBus`. Aplicatia inregistreaza `AuditLogObserver` ca observer global, deci acesta primeste toate evenimentele. Apoi inregistreaza `WelcomeEmailObserver` pentru evenimente de tip `CONTACT_CREATED`, `EnrollmentConfirmationObserver` pentru `ENROLLMENT_CREATED` si `LeadScoreUpdateObserver` pentru `ACTIVITY_COMPLETED`. Aceasta etapa arata ca autorul a separat actiunile principale de reactiile secundare. De exemplu, cand se creeaza un contact, serviciul nu trebuie sa stie in detaliu cum se trimite emailul de bun venit; el doar publica un eveniment, iar observerul potrivit reactioneaza.

La final, `CrmApplication` inregistreaza un shutdown hook. Acesta este un fir de executie care se activeaza cand JVM-ul se inchide si are rolul de a elibera resursele, in special connection pool-ul. Apoi se creeaza `ConsoleUI` si se apeleaza metoda `run`. Astfel, aplicatia intra intr-o bucla in care afiseaza meniul, citeste optiunea utilizatorului si trimite operatia mai departe catre facade.

4. Arhitectura pe straturi

Arhitectura proiectului este una stratificata. In partea de sus se afla UI-ul de consola, care are rolul de a interactiona cu utilizatorul. `ConsoleUI` nu ar trebui sa contina logica de business complicata. El citeste date, construiesc obiecte prin builderi si apeleaza metode din `CrmFacade`. Aceasta separare este importanta deoarece interfata poate fi schimbata ulterior. De exemplu, daca s-ar crea un REST API, acesta ar putea apela acelasi `CrmFacade` sau aceleasi servicii, fara sa rescrie logica de business.

`CrmFacade` este stratul care ascunde complexitatea sistemului. In loc ca UI-ul sa cunoasca toate serviciile, toate comenzile si toate repository-urile, el foloseste o interfata unica. Metode precum `createContact`, `getHotLeads`, `changeLeadStatus`, `createOpportunity` sau `logActivity` delega operatiile catre servicii si comenzi. Acesta este rolul pattern-ului Facade: sa ofere un punct de intrare simplu pentru un subsistem mai mare.

Sub facade se afla serviciile. Serviciile contin regulile de business: validare, verificare duplicate, calcul scor, schimbare status, calcul pret, emitere evenimente, completare valori implicite. De exemplu, `ContactService` nu se limiteaza la a salva un contact. El valideaza datele, verifica daca emailul exista deja, calculeaza scorul initial printr-o strategie, salveaza contactul si publica un eveniment. Acest lucru arata diferenta dintre un DAO si un service. DAO-ul lucreaza cu SQL, serviciul lucreaza cu reguli de business.

Mai jos sunt repository-urile si DAO-urile. Repository-ul ofera metode orientate pe domeniu, cum ar fi `findHotLeads`, `findByEmail`, `findByLeadStatus` sau `findActivePipeline`. DAO-ul este responsabil de SQL

concret: INSERT, UPDATE, SELECT, maparea ResultSet-ului in obiecte Java si setarea parametrilor in PreparedStatement. AbstractDao ofera scheletul comun al operatiilor CRUD, iar clasele concrete precum ContactDao, CourseDao sau OpportunityDao implementeaza detaliile specifice fiecarei tabele.

Pe langa fluxul vertical UI - Facade - Service - Repository - DAO - Database, exista componente laterale: EventBus pentru evenimente, CommandInvoker pentru comenzi, Strategy pentru algoritmi interschimbabili, Factory pentru creare de obiecte, Builder pentru construire fluenta si ValidatorChain pentru validare modulara. Aceste componente nu sunt independente de arhitectura, ci o completeaza.

Schema simplificata a fluxului

```
ConsoleUI -> CrmFacade -> CommandInvoker -> Service
Service -> Repository -> DAO -> Database
Service -> EventBus -> Observers
Service -> Strategy / Validator / Factory / Builder
```

5. Modelul de domeniu si entitatile principale

Modelul de domeniu se afla in pachetul crm.model.entity. Toate entitatile importante extind BaseEntity, care contine id, createdAt si updatedAt. Aceste campuri sunt comune pentru majoritatea tabelor si permit un comportament uniform. BaseEntity mai defineste metoda isNew, folosita de repository pentru a decide daca un obiect trebuie inserat sau actualizat. Daca id-ul este null, obiectul este nou si se face INSERT; daca id-ul exista, se face UPDATE.

Contact este cea mai importanta entitate. Ea contine campuri pentru persoane fizice: firstName, lastName si birthDate, dar si campuri pentru companii: companyName, fiscalCode, registrationNumber, industry si employeeCount. Campurile comune sunt email, phone, adresa, leadSource, leadStatus, leadScore, experienceLevel, learningGoal, assignedTo si consimtamantul GDPR. Metoda getFullName intoarce numele companiei pentru contacte corporate sau prenumele si numele pentru persoane individuale. Metodele isCorporate, isIndividual si isHotLead sunt metode de domeniu simple, dar utile pentru expresivitatea codului.

Course descrie oferta educationala. Contine cod, nume, descriere, programa, categorie, nivel, prerechizite, durata, pret individual, pret de grup, pret corporate pe zi si limite de participanti. CourseSession reprezinta o sesiune concreta a unui curs, cu data de inceput, data de final, trainer, mod de livrare, locatie, numar maxim de participanti si status. Diferenta dintre Course si CourseSession este importanta: un curs este produsul educational generic, iar sesiunea este instanta programata in timp.

Enrollment reprezinta inscrierea unui contact la o sesiune. Ea pastreaza contactId, sessionId, data inscrierii, status, pret, discount, pret final, statusul platii, suma platita, prezenta, nota finala, certificat si feedback. Metoda getRemainingAmount calculeaza cat mai are de platit cursantul. Aceasta metoda este un exemplu de logica de domeniu aflata direct in entitate, pentru ca depinde doar de datele proprii ale inscrierii.

Opportunity este modelul pentru pipeline-ul B2B. O oportunitate apartine unui client corporate, are titlu, descriere, numar estimat de participanti, cerinte personalizate, mod de livrare, valoare estimata, valoare ofertata, probabilitate, etapa si data estimata de inchidere. Metodele isClosed, isWon si getWeightedValue ajuta la interpretarea oportunitatii. getWeightedValue inmulteste valoarea ofertata cu probabilitatea si poate fi folosita pentru estimarea valorii reale a pipeline-ului.

Activity pastreaza interactiunile cu un contact sau cu o oportunitate: emailuri, apeluri, meeting-uri, task-uri, note si demo-uri. Activitatea are subiect, descriere, data programata, data finalizarii, durata, status, prioritate, rezultat si pasi urmasi. Metodele isCompleted si isOverdue sunt utile pentru a vedea daca o activitate este finalizata sau intarziata.

6. Contacte, lead-uri si calculul scorului

Fluxul de creare a unui contact incepe de obicei in ConsoleUI. Pentru un lead individual, UI-ul citeste prenumele, numele, emailul si telefonul, apoi foloseste ContactBuilder. Builderul seteaza contactType la INDIVIDUAL, seteaza numele, emailul, telefonul, sursa lead-ului, nivelul de experienta, obiectivul de invatare si consimtamantul GDPR. Pentru client corporate, acelasi builder seteaza contactType la

CORPORATE, companyName, fiscalCode, industry si employeeCount. Astfel, codul de creare devine fluent si usor de citit.

Dupa construire, contactul este trimis catre `crm.createContact`. `CrmFacade` nu salveaza direct contactul, ci creeaza o comanda `CreateContactCommand` si o trimite catre `CommandInvoker`. `CommandInvoker` apeleaza execute pe comanda si pastreaza comanda intr-un istoric limitat la 100 de elemente. In prezent, istoricul este folosit mai mult ca demonstratie, dar conceptul permite ulterior audit, cozi de executie sau undo.

`CreateContactCommand` apeleaza `ContactService.createContact`. Acolo are loc logica reala. Mai intai se ruleaza `ContactValidatorChain`. Lantul de validare include `EmailValidator`, `PhoneValidator`, `ContactTypeValidator` si `GdprValidator`. `EmailValidator` verifica existenta si formatul emailului. `PhoneValidator` verifica telefonul doar daca acesta este prezent, deoarece telefonul este optional. `ContactTypeValidator` verifica daca pentru persoane individuale exista nume sau prenume, iar pentru corporate exista numele companiei. `GdprValidator` verifica data consimtamantului daca `gdprConsent` este true.

Dupa validare, `ContactService` verifica daca emailul este deja inregistrat. Aceasta regula este importanta deoarece tabela contacts are email unic. Daca emailul exista, se arunca `BusinessException`. Daca totul este corect, serviciul seteaza statusul implicit NEW, `firstContactDate` si `lastContactDate`. Apoi intra in scena Strategy Pattern. `LeadScoringContext` alege strategia potrivita in functie de tipul contactului: pentru CORPORATE foloseste `B2BLeadScoringStrategy`, iar pentru INDIVIDUAL foloseste `SimpleLeadScoringStrategy`.

`SimpleLeadScoringStrategy` calculeaza scorul initial pe baza completitudinii datelor si a sursei lead-ului. Emailul, telefonul, numele, sursa, nivelul de experienta, obiectivul de invatare, consimtamantul GDPR si marketingul contribuie la scor. De exemplu, un lead venit prin referral primeste mai multe puncte decat unul venit prin cold call, iar un obiectiv de reconversie profesionala creste scorul. `B2BLeadScoringStrategy` se uita la factori specifici companiilor: date fiscale, industrie, numar de angajati si oras. O companie din IT sau software cu multi angajati primeste scor mai mare.

Dupa calcularea scorului, contactul este salvat prin `ContactRepository` si `ContactDao`. La final, `ContactService` publica `ContactCreatedEvent` in `EventBus`. Acest eveniment declanseaza doua efecte: `AuditLogObserver` il inregistreaza in `audit_logs`, iar `WelcomeEmailObserver` trimite un email de bun venit prin `NotificationService`. In realitate emailul nu este trimis prin SMTP, ci este logat. Chiar si asa, arhitectura arata cum s-ar integra ulterior un serviciu real de notificari.

7. Activitati si reactii automate prin EventBus

Activitatile reprezinta interactiuni intre echipa de vanzari si contacte. `ConsoleUI` permite crearea de emailuri, apeluri, meeting-uri si note. Pentru aceste obiecte se foloseste `ActivityFactory`. Factory-ul ascunde detaliile de creare si seteaza valori implicite potrivite. Un apel are de obicei durata de 30 de minute si prioritate HIGH, un email are prioritate MEDIUM, iar o nota este marcata direct ca COMPLETED. Aceasta abordare evita duplicarea valorilor implicite in UI sau in alte componente.

Cand o activitate este creata, `CrmFacade` apeleaza `ActivityService.createActivity`. Serviciul seteaza statusul implicit SCHEDULED si prioritatea implicita MEDIUM daca lipsesc, apoi salveaza activitatea prin `ActivityRepository`. La simpla creare nu se publica eveniment de scoring, deoarece o activitate programata nu inseamna neaparat interes confirmat. Scorul trebuie modificat atunci cand activitatea este finalizata sau cand exista un rezultat concret.

Finalizarea activitatii se face prin `ActivityService.completeActivity`. Serviciul cauta activitatea dupa id, seteaza statusul COMPLETED, `completedDate` la momentul curent, `outcome` si `nextSteps`, apoi salveaza modificarile. Dupa salvare, publica `ActivityCompletedEvent`. Acest eveniment este ascultat de `LeadScoreUpdateObserver`, care are rolul de a transforma activitatile finalizate in impact asupra scorului lead-ului.

LeadScoreUpdateObserver primește evenimentul și verifică dacă este într-adevăr ActivityCompletedEvent. Extrage activitatea, verifică dacă există contactId, apoi mapează tipul activității la un cod de scoring. EMAIL devine EMAIL_OPENED, CALL devine CALLBACK_REQUESTED, MEETING devine MEETING_ATTENDED, iar DEMO devine DEMO_REQUESTED. Apoi apelează ContactService.updateLeadScoreFromActivity. Această metodă recalculează impactul prin strategia potrivită și ajustează scorul între 0 și 100.

Acest mecanism este un exemplu bun de design event-driven la scară mică. Serviciul de activități nu știe regulile exacte de scoring și nu depinde direct de strategia de scoring. El doar anunță că o activitate a fost finalizată. Observerul specializat se ocupă de consecință. Astfel, dacă în viitor s-ar dori trimiterea unei notificări după un meeting, crearea automată a unui follow-up sau actualizarea unui dashboard, s-ar adăuga noi observatori fără să se modifice ActivityService.

8. Oportunități B2B și pipeline comercial

Pentru clienții corporate, proiectul modelează procesul de vânzare prin oportunități. O oportunitate reprezintă o discuție comercială concretă, de exemplu un training Java pentru o echipă de dezvoltatori. Entitatea Opportunity include clientId, titlu, descriere, număr estimat de participanți, cerințe personalizate, mod de livrare, locație preferată, data dorită de start, valoare estimată, valoare ofertată, probabilitate, etapă și data așteptată de închidere.

Crearea unei oportunități se face în UI prin OpportunityBuilder. Builderul setează clientul, titlul, numărul de participanți, valoarea estimată, modul de livrare, etapa și data estimată de închidere. Dacă etapa nu este setată explicit, constructorul builderului porneste de la LEAD_QUALIFICATION și folosește probabilitatea implicită pentru această etapă. Aceasta înseamnă că obiectul Opportunity începe într-o stare validă, chiar dacă utilizatorul nu introduce toate datele.

Pipeline-ul este controlat prin enumul OpportunityStage. Etapele sunt LEAD_QUALIFICATION, NEEDS_ANALYSIS, PROPOSAL_SENT, NEGOTIATION, CONTRACT_REVIEW, WON și LOST. Fiecare etapă are și o probabilitate implicită. De exemplu, LEAD_QUALIFICATION are probabilitate mai mică, iar CONTRACT_REVIEW are probabilitate mare. WON înseamnă 100%, LOST înseamnă 0%. Aceasta alegere permite sistemului să actualizeze automat probabilitatea când oportunitatea se mută dintr-o etapă în alta.

CrmFacade.createOpportunity folosește CreateOpportunityCommand, iar comanda apelează OpportunityService.createOpportunity. Serviciul verifică dacă există clientId și titlu, setează etapa și probabilitatea implicită dacă lipsesc, apoi salvează oportunitatea. Mutarea în pipeline se face prin MoveOpportunityStageCommand, care apelează OpportunityService.moveToStage. Această metodă citește oportunitatea, compară etapa veche cu etapa nouă, setează etapa și probabilitatea nouă, iar dacă etapa este WON sau LOST setează actualCloseDate.

După mutare, OpportunityService publică OpportunityStageChangedEvent. În prezent nu există un observer specific pentru acest eveniment, dar AuditLogObserver îl primește pentru că este observer global. Prin urmare, orice schimbare de etapă poate fi auditată. Modelul este pregătit pentru extinderi precum notificarea managerului când o oportunitate ajunge în NEGOTIATION, generarea unei propuneri când ajunge în PROPOSAL_SENT sau crearea unui contract când ajunge în WON.

9. Cursuri, sesiuni, înscrieri și strategii de pricing

Partea educatională a CRM-ului este reprezentată prin cursuri, sesiuni și înscrieri. Course descrie cursul ca produs: cod, nume, descriere, durată, categorie, nivel și prețuri. CourseSession descrie o sesiune concretă: perioada, trainer, locație, mod de livrare și număr de participanți. Enrollment leagă un contact de o sesiune și păstrează informații despre preț, discount, plată, participare, notă și certificat.

CourseService conține logica de bază pentru cursuri. La creare verifică dacă există codul cursului, dacă acel cod nu este duplicat, dacă există nume și dacă durata este pozitivă. Dacă lipsesc anumite valori, setează active la true, minParticipants la 3 și maxParticipants la 15. Apoi salvează cursul prin CourseRepository. Această logică este simplă, dar arată rolul serviciului: nu lasă DAO-ul să decida reguli de business.

EnrollmentService este mai interesant deoarece folosește Strategy Pattern pentru calculul prețului. Metoda enrollContact primește contactId, sessionId, un obiect Course și o strategie de pricing. Dacă

strategia lipseste, foloseste `IndividualPricingStrategy` fara `early bird` si fara plata integrala in avans. Inainte de creare verifica daca acel contact este deja inscris la aceeasi sesiune. Aceasta verificare evita dublarea inscrierilor si corespunde si cheii unice din schema SQL.

`IndividualPricingStrategy` calculeaza pretul pentru cursanti individuali. Ea poate aplica discount `early bird` si discount pentru plata integrala. `CorporatePricingStrategy` calculeaza pretul pentru clienti corporate pornind de la pretul pe zi si de la durata cursului in ore, apoi aplica discount in functie de numarul de participanti. Pentru grupuri mari discountul este mai mare. Avantajul Strategy Pattern este ca `EnrollmentService` nu trebuie sa contina toate regulile de pricing. El primeste o strategie si o foloseste prin interfata `PricingStrategy`.

Dupa crearea inscrierii, `EnrollmentService` schimba statusul contactului la `ENROLLED` prin `ContactService.updateLeadStatus`. Apoi publica `EnrollmentCreatedEvent`. `EnrollmentConfirmationObserver` asculta acest eveniment, cauta contactul asociat si trimite confirmarea de inscriere prin `NotificationService`. `NotificationService` creeaza notificari prin `EmailNotificationFactory` si, daca exista telefon, prin `SmsNotificationFactory`. In implementarea actuala notificarea este doar logata, dar structura permite integrarea ulterioara cu SMTP sau Twilio.

`EnrollmentService` mai are doua operatii importante. `registerPayment` adauga suma platita si schimba `paymentStatus` la `PAID` sau `PARTIAL` in functie de total. Daca suma platita acopera pretul final, inscrierea devine `CONFIRMED`. `completeEnrollment` marcheaza inscrierea ca finalizata, seteaza nota, decide daca examenul a fost promovat si genereaza un numar de certificat daca nota este cel putin 5. Astfel, fluxul educational este modelat de la inscriere pana la certificare.

10. Persistenta: Repository, DAO si JDBC

Accesul la date este impartit in doua niveluri: `Repository` si `DAO`. `Repository`-ul este mai aproape de limbajul domeniului, iar `DAO`-ul este mai aproape de SQL. De exemplu, `ContactRepository` expune metode precum `findByEmail`, `findByLeadStatus`, `findHotLeads`, `findByAssignedTo`, `search`, `findStaleLeads` si `countByLeadStatus`. `ContactDao` implementeaza efectiv interogari SQL pentru aceste metode. Aceasta separare ajuta la mentinerea codului de business curat, deoarece serviciile lucreaza cu repository-uri, nu direct cu `PreparedStatement`.

`AbstractRepository` contine operatii generale pentru entitati: `save`, `findById`, `getById`, `findAll`, `count`, `deleteById` si `existsById`. Metoda `save` verifica `entity.isNew`. Daca id-ul este null, apeleaza `DAO.save`; altfel, `DAO.update`. Aceasta decizie este utila deoarece serviciile nu trebuie sa stie daca obiectul este nou sau existent. Tot aici apare `ResourceNotFoundException`, aruncata cand `getById` nu gaseste entitatea.

`AbstractDao` este una dintre cele mai importante clase tehnice. Ea implementeaza `Template Method Pattern`. Clasa abstracta defineste algoritmul general pentru `save`, `update`, `findById`, `findAll`, `count`, `deleteById` si `existsById`, dar lasa subclasele sa defineasca detaliile: numele tabeli, SQL-ul de insert, SQL-ul de update, maparea `ResultSet`-ului, setarea parametrilor si setarea id-ului generat. Astfel, codul repetitiv JDBC este centralizat, iar clasele concrete se concentreaza pe particularitatile entitatii.

`ContactDao` este cel mai complex DAO. El construiesc un `INSERT` cu toate campurile relevante din tabela `contacts`, seteaza parametrii in ordine, transforma enumerile in stringuri si converteste `LocalDate` sau `LocalDateTime` in tipuri SQL. La citire, `mapResultSetToEntity` reconstruieste obiectul `Contact` din coloanele tabeli. Pentru enumuri foloseste `parseEnum`, care intoarce null daca valoarea din baza nu corespunde enumului. Aceasta alegere evita crash-uri in unele situatii, dar poate ascunde date invalide.

`CourseDao`, `EnrollmentDao`, `OpportunityDao` si `ActivityDao` urmeaza acelasi model. Fiecare defineste SQL pentru insert si update, mapeaza rezultatele si adauga interogari specifice. `OpportunityDao`, de exemplu, are `findByClientId`, `findByStage` si `findActivePipeline`. `ActivityDao` are `findById`, `findByOpportunityId` si `findUpcoming`. Aceste metode reflecta intrebari reale pe care un CRM trebuie sa le raspunda: ce activitati are un contact, ce oportunitati sunt active, ce cursuri sunt disponibile sau cine nu a platit.

Un aspect important este ca proiectul foloseste `PreparedStatement`, deci parametrii sunt legati sigur si nu concatenati direct in SQL, cu exceptia numelui tabeli controlat de cod. Aceasta reduce riscul de SQL

injection pentru valorile introduse de utilizator. Totusi, nu toate operatiile compuse sunt tranzactionale. De exemplu, la creare contact se salveaza contactul si apoi se publica evenimente, iar observatorii pot scrie in audit_logs. Daca una dintre operatiile secundare esueaza, sistemul logheaza eroarea, dar nu ruleaza o tranzactie comuna pentru tot fluxul.

11. Design patterns si rolul lor in proiect

Proiectul este construit explicit pentru a demonstra design patterns. Singleton este cel mai raspandit. AppConfig garanteaza o singura sursa de configurare, DatabaseConnection garanteaza un singur connection pool, EventBus garanteaza un singur canal de evenimente, iar serviciile, repository-urile si DAO-urile sunt accesate prin getInstance. Avantajul este simplitatea pentru o aplicatie mica de consola. Dezavantajul este ca testarea devine mai dificila, deoarece dependintele sunt globale si greu de inlocuit cu mock-uri.

Facade Pattern este reprezentat de CrmFacade. Aceasta clasa ascunde complexitatea subsistemelor. UI-ul nu trebuie sa stie daca o operatie foloseste o comanda, un serviciu, un repository sau un observer. Pentru utilizatorul facade-ului, createContact inseamna pur si simplu creare contact. Intern insa se activeaza validare, scoring, persistenta si evenimente. Acest pattern este potrivit pentru proiect deoarece exista multe componente si o interfata simpla de demo.

Command Pattern apare prin interfata Command, AbstractCommand, CommandInvoker si comenziile concrete. O comanda incapsuleaza o operatie business ca obiect. CreateContactCommand, UpdateLeadStatusCommand, EnrollContactCommand, CreateOpportunityCommand si MoveOpportunityStageCommand sunt exemple clare. AbstractCommand adauga logging si masurarea duratei printr-un Template Method: execute este final, iar subclasele implementeaza doExecute. CommandInvoker pastreaza istoricul comenzilor, ceea ce ar permite audit sau undo in viitor.

Observer Pattern este implementat prin EventBus, CrmEvent si observeri. Serviciile publica evenimente, iar observerii reactioneaza. ContactCreatedEvent declanseaza welcome email, EnrollmentCreatedEvent declanseaza confirmarea inscrierii, ActivityCompletedEvent declanseaza actualizarea scorului, iar AuditLogObserver primeste toate evenimentele. Acest pattern reduce cuplarea intre actiunea principala si efectele secundare.

Strategy Pattern este folosit pentru lead scoring si pricing. LeadScoringStrategy are implementari diferite pentru B2C si B2B. PricingStrategy are implementari pentru cursanti individuali si corporate. Aceasta permite schimbarea algoritmului fara rescrierea serviciului care il foloseste. De exemplu, daca firma ar dori o strategie avansata bazata pe comportament web, ar adauga o noua clasa care implementeaza LeadScoringStrategy.

Builder Pattern apare in ContactBuilder si OpportunityBuilder. Chiar daca Lombok genereaza builderi generici, autorul a creat builderi custom pentru a avea metode expresive precum asIndividual, asCorporate, withGdprConsent, participants sau expectedClose. Acesti builderi seteaza valori implicite si valideaza minimal obiectele inainte de build. Factory Pattern este folosit pentru activitati si notificari. Chain of Responsibility este folosit pentru validarea contactelor, iar Template Method apare in AbstractDao si AbstractCommand.

Rezumat al pattern-urilor

Privit in ansamblu, proiectul foloseste pattern-urile pentru a separa intentia de implementare. Facade ascunde complexitatea, Command transforma actiunile in obiecte, Observer decupleaza efectele secundare, Strategy muta algoritmi in clase interschimbabile, iar Repository si DAO separa limbajul domeniului de SQL. Aceasta este ideea arhitecturala care trebuie retinuta: fiecare pattern are un loc practic in fluxul aplicatiei.

12. Validare, exceptii si logging

Validarea este tratata mai serios pentru contacte decat pentru alte entitati. ContactValidatorChain construiesc un lant de validatori: EmailValidator, PhoneValidator, ContactTypeValidator si GdprValidator. Fiecare validator primeste entitatea si o lista de erori. Validatorul curent adauga erorile sale, apoi delega

mai departe catre urmatorul. La final, ContactValidatorChain poate returna lista de erori sau poate arunca ValidationException. Aceasta structura permite adaugarea usoara a unor reguli noi, de exemplu validarea codului fiscal pentru companii.

Exceptiile custom sunt simple, dar utile. CrmException este clasa de baza runtime. BusinessException indica probleme de business, cum ar fi email duplicat sau lipsa unui camp obligatoriu. DataAccessException indica probleme la baza de date sau la SQL. ResourceNotFoundException este aruncata cand o entitate cautata dupa id nu exista. ValidationException contine o lista de erori de validare si construiește mesajul prin concatenarea lor. Aceasta ierarhie face codul mai expresiv decat folosirea directa a RuntimeException peste tot.

Logging-ul este prezent in majoritatea componentelor importante. CrmApplication logheaza pornirea si oprirea. DatabaseConnection logheaza initializarea si inchiderea connection pool-ului. Serviciile logheaza operatii de business, precum creare contact, creare curs, inscriere, plata sau mutare oportunitate. CommandInvoker si AbstractCommand logheaza executia comenzilor. EventBus logheaza inregistrarea observerilor si notificarea evenimentelor. DAO-urile logheaza erori SQL si operatii de salvare sau update la nivel debug.

Din punct de vedere didactic, logurile sunt foarte valoroase deoarece arata fluxul real. De exemplu, cand se creeaza un contact, logul poate arata invocarea comenzii, calculul scorului initial, salvarea in tabela contacts, publicarea evenimentului, trimiterea simulata a emailului si finalizarea comenzii. Astfel, se poate demonstra functionarea arhitecturii fara debugger.

Exista totusi zone unde validarea ar putea fi extinsa. Oportunitatile verifica doar clientId si titlul. Inscrierile verifica dublura pe sesiune, dar nu verifica daca sesiunea este plina sau daca exista locuri disponibile. Platile nu verifica explicit daca suma este pozitiva. Activitatile nu valideaza obligatoriu contactId, subiect sau tip. Acestea sunt limitari normale pentru un proiect demonstrativ, dar pot fi mentionate ca directii de imbunatatire.

13. Baza de date, schema SQL si observatii de consistenta

Proiectul include doua surse SQL: sql/schema.sql si src/main/resources/schema.sql. Din analiza codului DAO, schema care corespunde cel mai bine implementarii este sql/schema.sql. Aceasta contine coloane precum courses.name, price_individual, price_group, opportunities.client_id si probability_percent, exact asa cum le folosesc CourseDao si OpportunityDao. Schema din src/main/resources foloseste in unele locuri alte denumiri, precum courses.title, base_price, opportunities.contact_id si probability. Aceasta diferenta este importanta deoarece, daca se ruleaza schema gresita, aplicatia poate porni, dar operatiile vor esua la runtime.

Configuratia AppConfig contine proprietatea db.init.schema=true, dar in cod nu exista o componenta care sa citeasca automat schema.sql si sa creeze tabelele. URL-ul JDBC are createDatabaseIfNotExist=true, ceea ce poate crea baza crm_training, dar nu creeaza tabelele. Logurile proiectului confirma aceasta problema: intr-o rulare, aplicatia s-a conectat la MariaDB, dar la listarea lead-urilor a aparut eroarea Table crm_training.contacts doesn't exist. Concluzia este ca schema trebuie rulata manual inainte de pornirea aplicatiei.

Schema sql/schema.sql este mai completa decat functionalitatea expusa in UI. Ea contine nu doar contacte, cursuri, trainerii, sesiuni, inscrieri, oportunitati si activitati, ci si proposals, contracts, invoices, payments, lead_score_logs si audit_logs. Asta arata ca autorul a gandit un CRM mai mare decat demo-ul din consola. Unele tabele sunt pregatite pentru extensii, chiar daca in cod nu exista inca servicii complete pentru ele.

Indexurile din schema sunt alese in functie de interogariile frecvente. Emailul este indexat pentru cautarea rapida a contactelor. lead_status, assigned_to si contact_type ajuta la filtrari. courses are index pe code si category. opportunities are index pe stage, iar activities pe activity_type si scheduled_date. Aceste alegeri reflecta cerintele CRM: cautare dupa email, listare lead-uri dupa status, listare oportunitati dupa etapa si gasirea activitatilor programate.

Auditul este realizat prin tabela `audit_logs` si `AuditLogObserver`. Observerul incearca sa extraga id-ul entitatii prin reflexie, apeland `getId` pe `payload`. Apoi insereaza `entity_type`, `entity_id`, `action` si `new_values`. Aceasta implementare este generica si functioneaza pentru toate evenimentele care au `payload` cu `id`. Totusi, in forma actuala, auditul salveaza `payload.toString`, ceea ce poate fi greu de comparat sau de analizat automat. Intr-o versiune mai avansata, ar fi mai bine sa se salveze JSON cu valori vechi si noi.

14. Cerinte functionale si nefunctionale deduse din cod

Chiar daca proiectul nu are un document separat de requirements, cerintele pot fi deduse din meniu, servicii, modele si schema SQL. Cerintele functionale principale sunt: gestionarea contactelor B2C si B2B, cautarea contactelor dupa email, calcularea si actualizarea scorului de lead, listarea lead-urilor fierbinti, schimbarea statusului unui lead, gestionarea activitatilor, gestionarea oportunitatilor B2B, vizualizarea pipeline-ului, gestionarea cursurilor active, inscrierea contactelor la cursuri, inregistrarea platilor si finalizarea inscrierilor cu certificat.

Pentru contactele B2C, sistemul trebuie sa stocheze nume, email, telefon, sursa lead-ului, nivelul de experienta si obiectivul de invatare. Pentru contactele B2B, sistemul trebuie sa stocheze numele companiei, cod fiscal, industrie si numar de angajati. Pentru ambele tipuri, sistemul pastreaza status, scor, activitati si consimtamant GDPR. Aceasta cerinta reflecta ideea ca o firma de training are atat clienti individuali, cat si clienti corporate, dar vrea sa ii urmareasca intr-un sistem comun.

Cerintele nefunctionale sunt la fel de vizibile. Separarea pe straturi indica o cerinta de mentenabilitate. Folosirea repository-urilor si DAO-urilor indica o cerinta de separare a business-ului de SQL. Folosirea HikariCP indica o preocupare pentru eficienta conexiunilor la baza de date. Logging-ul indica o cerinta de observabilitate. Singleton-urile simplifica accesul la resurse comune intr-o aplicatie de consola. Observerii indica o cerinta de extensibilitate pentru efecte secundare.

Securitatea este prezenta partial. Schema are `password_hash` pentru utilizatori, dar UI-ul nu implementeaza login. `PreparedStatement` reduce riscul de SQL injection. GDPR este modelat prin campuri de consimtamant si validare. Totusi, nu exista control de acces, criptare, protectie pentru parole in configuratie sau management al sesiunii. Aceste aspecte pot fi mentionate ca limitari ale proiectului in forma actuala.

Scalabilitatea este suficienta pentru o aplicatie locala sau demo. HikariCP permite folosirea eficienta a conexiunilor, iar indexurile ajuta interogările. Totusi, arhitectura nu este distribuita, nu exista cache, nu exista API stateless si Singleton-urile globale pot deveni problematice in testare sau in aplicatii cu mai multe instante. Pentru scopul proiectului de licenta, aceste alegeri sunt acceptabile, deoarece accentul este pe claritatea arhitecturii si pe aplicarea pattern-urilor.

15. Limitari si posibile imbunatatiri

Prima limitare importanta este diferenta dintre cele doua fisiere de schema SQL. Pentru ca aplicatia sa fie stabila, ar trebui pastrata o singura schema oficiala sau ar trebui introdus un sistem de migrari, de exemplu Flyway sau Liquibase. In acest mod, baza de date ar fi creata si actualizata automat, iar riscul ca DAO-urile sa nu se potriveasca cu tabelele ar scadea semnificativ.

A doua limitare este lipsa testelor automate. Desi `pom.xml` include JUnit 5, nu exista teste in `src/test`. Pentru un proiect cu atat de multe reguli, ar fi utile teste unitare pentru strategiile de scoring, strategiile de pricing, validatorii, builderii si serviciile. Ar fi utile si teste integration pentru DAO-uri, folosind o baza de date de test. In prezent, corectitudinea se verifica prin rulare manuala si prin loguri.

A treia limitare este cuplarea prin Singleton-uri. Pentru un proiect didactic, Singleton simplifica mult codul, dar pentru testare si productie poate deveni incomod. Serviciile isi creeaza singure repository-urile, repository-urile isi creeaza singure DAO-urile, iar DAO-urile folosesc direct `DatabaseConnection`. O imbunatatire ar fi introducerea dependency injection, manual sau printr-un framework precum Spring. Astfel, componentele ar putea fi inlocuite mai usor in teste.

A patra limitare este lipsa unei interfete moderne. ConsoleUI este potrivita pentru demo, dar un CRM real ar avea cel putin un REST API si o interfata web. Ar fi utile ecrane pentru lista de contacte, detaliu contact, timeline activitati, pipeline Kanban, lista cursuri si dashboard de statistici. Logica existenta ar putea fi pastrata, iar UI-ul web ar consuma metodele din servicii sau dintr-un layer de controller.

A cincea limitare este lipsa tranzactiilor explicite pentru fluxuri compuse. De exemplu, la inscriere se creeaza enrollment, se schimba statusul contactului si se publica eveniment. Daca una dintre operatii esueaza, nu exista o tranzactie care sa garanteze rollback complet. Pentru productie, operatiile de business importante ar trebui executate in tranzactii clare.

Alte imbunatatiri posibile sunt integrarea reala cu email si SMS, generarea de facturi, folosirea tabelelor proposals si contracts, autentificare cu roluri, validare mai stricta pentru plati si sesiuni, export de rapoarte, dashboard cu indicatori de vanzari si curatarea encoding-ului romanesc din fisierele existente.

16. Concluzie

Codul proiectului CRM Training IT functioneaza ca un exemplu de aplicatie Java stratificata, orientata spre un domeniu de business realist. El porneste de la o nevoie clara: gestionarea lead-urilor, clientilor corporate, activitatilor, cursurilor, oportunitatilor si inscrierilor intr-o firma de training IT. In loc sa implementeze totul intr-o singura clasa sau intr-un CRUD simplu, autorul a separat responsabilitatile in UI, facade, comenzi, servicii, repository-uri, DAO-uri, modele, strategii, observeri si validatori.

Cea mai importanta idee arhitecturala este ca UI-ul nu lucreaza direct cu baza de date. O actiune din meniu trece prin CrmFacade si ajunge la un service. Service-ul aplica reguli de business si foloseste repository-ul. Repository-ul foloseste DAO-ul, iar DAO-ul executa SQL prin JDBC si HikariCP. Evenimentele sunt publicate separat prin EventBus, iar observerii se ocupa de audit, notificari si actualizari automate ale scorului. Aceasta structura face codul mai usor de explicat, extins si inteles.

Din perspectiva design patterns, proiectul este foarte bogat. Singleton gestioneaza resurse comune, Facade simplifica accesul, Command incapsuleaza operatii, Observer decupleaza efectele secundare, Strategy permite algoritmi interschimbabili, Builder simplifica obiectele complexe, Factory creeaza activitati si notificari, Chain of Responsibility organizeaza validarea, iar Template Method reduce duplicarea in DAO-uri. Aceste pattern-uri nu sunt doar mentionate, ci sunt legate de fluxuri concrete ale aplicatiei.

Proiectul are si limitari: schema SQL nu este complet unificata, initializarea bazei nu este automata, testele lipsesc, notificarea este simulata, autentificarea nu este implementata, iar UI-ul este doar consola. Totusi, pentru un proiect de licenta, aceste limitari nu anuleaza valoarea arhitecturala. Dimpotriva, ele pot fi prezentate ca directii naturale de dezvoltare. In forma actuala, proiectul demonstreaza un nucleu functional de CRM si o intelegere buna a separarii responsabilitatilor in aplicatii Java.

In concluzie, CRM Training IT este un proiect care trebuie explicat prin fluxuri, nu doar prin clase. Cand se creeaza un contact, sistemul valideaza datele, calculeaza scorul, salveaza in baza de date si declanseaza notificari. Cand se finalizeaza o activitate, sistemul actualizeaza scorul lead-ului. Cand se creeaza o oportunitate, ea intra intr-un pipeline cu etape si probabilitati. Cand se face o inscriere, sistemul calculeaza pretul, schimba statusul contactului si trimite confirmare. Aceste fluxuri arata cum codul transforma cerintele unui CRM intr-o implementare modulara si extensibila.